

Task 4: Database Implementation Report

1. Data Definition Language (DDL) Script

The following SQL script defines the physical schema, enforces data integrity through constraints, and implements business logic via triggers. This script is optimized for compatibility with both MariaDB and MySQL Workbench.

SQL

```
-- =====
-- DATABASE INITIALIZATION
-- =====

CREATE DATABASE IF NOT EXISTS tripistry;
USE tripistry;

-- =====
-- SUPERTYPE: User
-- =====
-

=====

CREATE TABLE User (
    userID INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(150) NOT NULL UNIQUE,
    passwordHash VARCHAR(255) NOT NULL,
    phone VARCHAR(25),
    joinDate DATE DEFAULT (CURRENT_DATE),
    role ENUM('traveller', 'agency', 'admin') NOT NULL
) ENGINE=InnoDB;

-- =====
-- SUBTYPES: Traveller & TravelAgency
-- =====

CREATE TABLE Traveller (
    userID INT PRIMARY KEY,
    firstName VARCHAR(60) NOT NULL,
    lastName VARCHAR(60) NOT NULL,
    passportNum VARCHAR(30),
    nationality VARCHAR(80),
    DOB DATE,
    CONSTRAINT fk_traveller_user FOREIGN KEY (userID) REFERENCES
User(userID) ON DELETE CASCADE
) ENGINE=InnoDB;
```

```

CREATE TABLE TravelAgency (
    userID INT PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    description TEXT,
    website VARCHAR(200),
    address VARCHAR(300),
    priceRange VARCHAR(10),
    CONSTRAINT fk_agency_user FOREIGN KEY (userID) REFERENCES
User(userID) ON DELETE CASCADE
) ENGINE=InnoDB;

-- =====
-- TRAVEL CONTENT & PACKAGES
-- =====

CREATE TABLE TravelPackage (
    packageID INT AUTO_INCREMENT PRIMARY KEY,
    agencyUserID INT NOT NULL,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    basePrice DECIMAL(10,2) NOT NULL CHECK (basePrice >= 0),
    durationDays INT NOT NULL CHECK (durationDays > 0),
    status ENUM('active', 'inactive', 'draft') DEFAULT 'draft',
    CONSTRAINT fk_package_agency FOREIGN KEY (agencyUserID) REFERENCES
TravelAgency(userID)
) ENGINE=InnoDB;

CREATE TABLE GroupTrip (
    packageID INT PRIMARY KEY,
    minGroupSize INT NOT NULL DEFAULT 2 CHECK (minGroupSize >= 2),
    maxGroupSize INT NOT NULL,
    currentGroupSize INT NOT NULL DEFAULT 0 CHECK (currentGroupSize >=
0),
    groupDeadline DATE,
    groupDescription TEXT,
    CONSTRAINT fk_grouptrip_package FOREIGN KEY (packageID) REFERENCES
TravelPackage(packageID) ON DELETE CASCADE,
    -- Table-level constraint for multi-column comparison
    CONSTRAINT chk_group_size_logic CHECK (maxGroupSize >= minGroupSize)
) ENGINE=InnoDB;

-- =====
-- TRANSACTIONS: Bookings & Reviews
-- =====

CREATE TABLE Booking (
    bookingID INT AUTO_INCREMENT PRIMARY KEY,

```

```

    travellerUserID INT NOT NULL,
    packageID INT NOT NULL,
    bookingDate DATE DEFAULT (CURRENT_DATE),
    numTravellers INT NOT NULL DEFAULT 1 CHECK (numTravellers > 0),
    totalAmount DECIMAL(12,2) NOT NULL,
    status ENUM('pending', 'confirmed', 'cancelled') DEFAULT 'pending',
    paymentStatus ENUM('unpaid', 'paid', 'refunded') DEFAULT 'unpaid',
    CONSTRAINT fk_booking_traveller FOREIGN KEY (travellerUserID)
REFERENCES Traveller(userID),
    CONSTRAINT fk_booking_package FOREIGN KEY (packageID) REFERENCES
TravelPackage(packageID)
) ENGINE=InnoDB;

CREATE TABLE Review (
    reviewID INT AUTO_INCREMENT PRIMARY KEY,
    travellerUserID INT NOT NULL,
    rating DECIMAL(3,2) NOT NULL CHECK (rating >= 0 AND rating <= 5),
    comment TEXT,
    reviewDate DATE DEFAULT (CURRENT_DATE),
    targetType ENUM('package', 'agency') NOT NULL,
    packageID INT NULL,
    agencyUserID INT NULL,
    CONSTRAINT fk_review_traveller FOREIGN KEY (travellerUserID)
REFERENCES Traveller(userID),
    CONSTRAINT fk_review_package FOREIGN KEY (packageID) REFERENCES
TravelPackage(packageID),
    CONSTRAINT fk_review_agency FOREIGN KEY (agencyUserID) REFERENCES
TravelAgency(userID)
) ENGINE=InnoDB;

-- =====
-- TRIGGERS: Business Logic Enforcement
-- =====

-- 1. Ensure Traveller Subtype matches User Role
DROP TRIGGER IF EXISTS trg_traveller_role_check;
DELIMITER $$
CREATE TRIGGER trg_traveller_role_check
BEFORE INSERT ON Traveller
FOR EACH ROW
BEGIN
    DECLARE v_role VARCHAR(20);
    SELECT role INTO v_role FROM User WHERE userID = NEW.userID;
    IF v_role <> 'traveller' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User must have
traveller role in User table.';

```

```

        END IF;
    END$$
DELIMITER ;

-- 2. Review XOR Logic (Package vs Agency)
DROP TRIGGER IF EXISTS trg_review_target_check;
DELIMITER $$
CREATE TRIGGER trg_review_target_check
BEFORE INSERT ON Review
FOR EACH ROW
BEGIN
    IF NOT (
        (NEW.targetType = 'package' AND NEW.packageID IS NOT NULL AND
        NEW.agencyUserID IS NULL) OR
        (NEW.targetType = 'agency' AND NEW.agencyUserID IS NOT NULL AND
        NEW.packageID IS NULL)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Review must target either a package or
an agency, not both or neither.';
    END IF;
END$$
DELIMITER ;

```

2. Implementation Explanations

A. Supertype/Subtype Specialization

The ERD defines a "Generalization" relationship for Users. This was implemented using the **Table-per-Subtype** approach.

- **Shared Identity:** Both **Traveller** and **TravelAgency** use **userID** as their Primary Key, which is also a Foreign Key referencing the **User** table. This ensures a strict 1:1 relationship and prevents a single entity from existing as a subtype without a corresponding supertype.
- **Role Enforcement:** The **trg_traveller_role_check** trigger ensures the integrity of this specialization. It prevents a user assigned an 'agency' role in the **User** table from being incorrectly inserted into the **Traveller** table.

B. Table-Level Check Constraints

To satisfy Task 4's requirement for data validation, several **CHECK** constraints were utilized.

- **Logical Comparisons:** In the `GroupTrip` table, the constraint `chk_group_size_logic` ensures that `maxGroupSize` is never less than `minGroupSize`.
- **Compliance:** This was defined at the table level (at the bottom of the `CREATE` statement) to comply with MySQL 8.0 standards, which forbid column-level checks from referencing other columns in the same row.

C. Polymorphic Association (XOR Constraint)

The `Review` table presented a unique challenge: it must relate to *either* a Travel Agency *or* a Travel Package.

- **The Solution:** We implemented two nullable foreign keys (`packageID` and `agencyUserID`).
- **Integrity Enforcement:** The `trg_review_target_check` trigger acts as an **Exclusive OR (XOR)** constraint. It validates that for every insert, exactly one of the two foreign keys is populated based on the `targetType`. This prevents "orphaned" reviews or reviews that target both entities simultaneously.

D. Data Population Strategy

To ensure the database is production-ready and satisfies the minimum requirements, a Python-based **Data Loader** was utilized to generate **500+ valid entries**.

- **Referential Integrity:** The loader script sequences inserts (User -> Subtype -> Package -> Booking) to ensure that all foreign key dependencies are satisfied during the population process.
- **Realism:** Using the `Faker` library, the dataset includes realistic emails, phone numbers, and dates that respect the `CHECK` constraints defined in the schema.

3. MySQLWorkbench Diagram

